

CONHECIMENTO E RACIOCÍNIO

3 – Raciocínio Lógico

Sistemas Periciais

Expert System (ES)



Raciocínio Lógico

- Sistemas de raciocínio baseado em regras IF ... THEN
- Tentam imitar a forma como o ser humano especialista numa área resolvem um problema
- Modelos mais usados de raciocínio lógico são os **Sistemas periciais**
 - sistemas de raciocínio lógico que se tornaram muito populares nos anos 80

Raciocínio Lógico

Sistemas Periciais

- Um **sistema pericial** é um programa que reproduz o raciocínio de um especialista numa determinada área.
- Um **sistema pericial** deve ser usado como um apoio à decisão de um perito numa determinada área, e nunca substitui o mesmo

Raciocínio Lógico

Sistemas Periciais

- Conhecimento:
 - **ESPECIALISTA:** alguém que possui um entendimento teórico e prático sobre uma determinada área ou domínio
 - O conhecimento para um sistema pericial é obtido através de um especialista de uma determinada área
 - **REGRAS:** o conhecimento extraído de um especialista é traduzido em regras lógicas do tipo
 - SE (antecedente) ENTÃO (consequente)
 - Antecedentes e consequentes: formados por conjunção ou disjunção de várias proposições lógicas
 - Base de conhecimento: formada pelo conjunto de regras

Raciocínio Lógico

Sistemas Periciais - Marcos históricos

- DENDRAL (1965): considerado o primeiro sistema pericial
 - Desenvolvido por Edward Feigenbaum e o especialista Joshua Lederberg
- MYCIN (1972):
 - Desenvolvido por Edward Shortliffe
 - 450 regras
- PROSPECTOR (1974-1983)
 - 1000 regras
- XCON (1978 - 1990)
 - Mais de 10000 regras
- CADUCEUS (1972 - 1986)
 - Diagnóstico médico em mais de 10000 patologias.

Raciocínio Lógico

Sistemas Periciais

- Um pouco de história
 - DENDRAL (1965): considerado o primeiro sistema pericial
 - Sistema implementado em LISP na Universidade de Stanford
 - Sistema que permitia obter respostas automáticas na área da química orgânica, sendo capaz de identificar estruturas moleculares de várias substâncias
 - Desenvolvido por Edward Feigenbaum e o especialista Joshua Lederberg

Raciocínio Lógico

Sistemas Periciais

- Um pouco de história
 - MYCIN (1972):
 - Sistema implementado em LISP na Universidade de Stanford
 - Sistema que permitia fazer diagnóstico médico para tratar infecções no sangue
 - Composto por cerca de 450 regras
 - Apesar do grande impacto científico nunca foi usado na prática devido a razões éticas
 - Desenvolvido por Edward Shortliffe

Raciocínio Lógico

Sistemas Periciais

- PROSPECTOR (1974-1983)
 - Implementado em LISP
 - Implementado com o auxílio de geólogos era capaz de localizar depósitos de minérios e identificar localizações ideais para extração e perfuração
 - Desenvolvido por uma equipa de 9 cientistas do Stanford Research Institute
 - Composto por cerca de 1000 regras

Raciocínio Lógico

Sistemas Periciais

- XCON (1978 - 1990):
 - Sistema implementado em OPS5
 - Capaz de selecionar componentes de forma otimizada para sistemas computacionais, baseados nas especificações dos utilizadores

Desenvolvido inicialmente por John P. McDermott (Carnegie Mellon University)

- Composto inicialmente por poucas regras, foi bastante popular, utilizado pela DEC (Digital Equipment Plant) com enorme sucesso comercial
- Atingiu nas suas versões finais mais de 10000 regras

Raciocínio Lógico

Sistemas Periciais

- CADUCEUS (1972 - 1986):
 - Sistema implementado em LISP
 - Desenvolvido por Harry Pople da Universidade de Pittsburgh com a colaboração de vários médicos da mesma universidade.
 - Capaz de efetuar diagnóstico médico de mais de 10000 patologias
 - Tentativa de recuperar a ideia pioneira do MYCIN

Raciocínio Lógico

Sistemas Periciais

- Outros Sistemas Periciais que ficaram na história:
 - HEARSY I, II, III (anos 70 com várias versões ainda hoje usadas)
– aconselhamento legal
 - MACSYMA (1968 – 1982) – manipulação de expressões matemáticas.
 - INTERNIST (1972) – diagnóstico médico
 - PUFF (1977) – avaliação da função pulmonar

Raciocínio Lógico

Sistemas Periciais

- Áreas de aplicação:
 - Áreas onde a base de conhecimento pode ser bem caracterizada por um especialista humano e facilmente transmitida a outras pessoas:
 - Contabilidade / Finanças
 - Diagnóstico médico
 - Controlo de processos em fabricas
 - Análise de risco em seguradoras, banca, etc\
 - Aconselhamento de compras
 - ...

Raciocínio Lógico

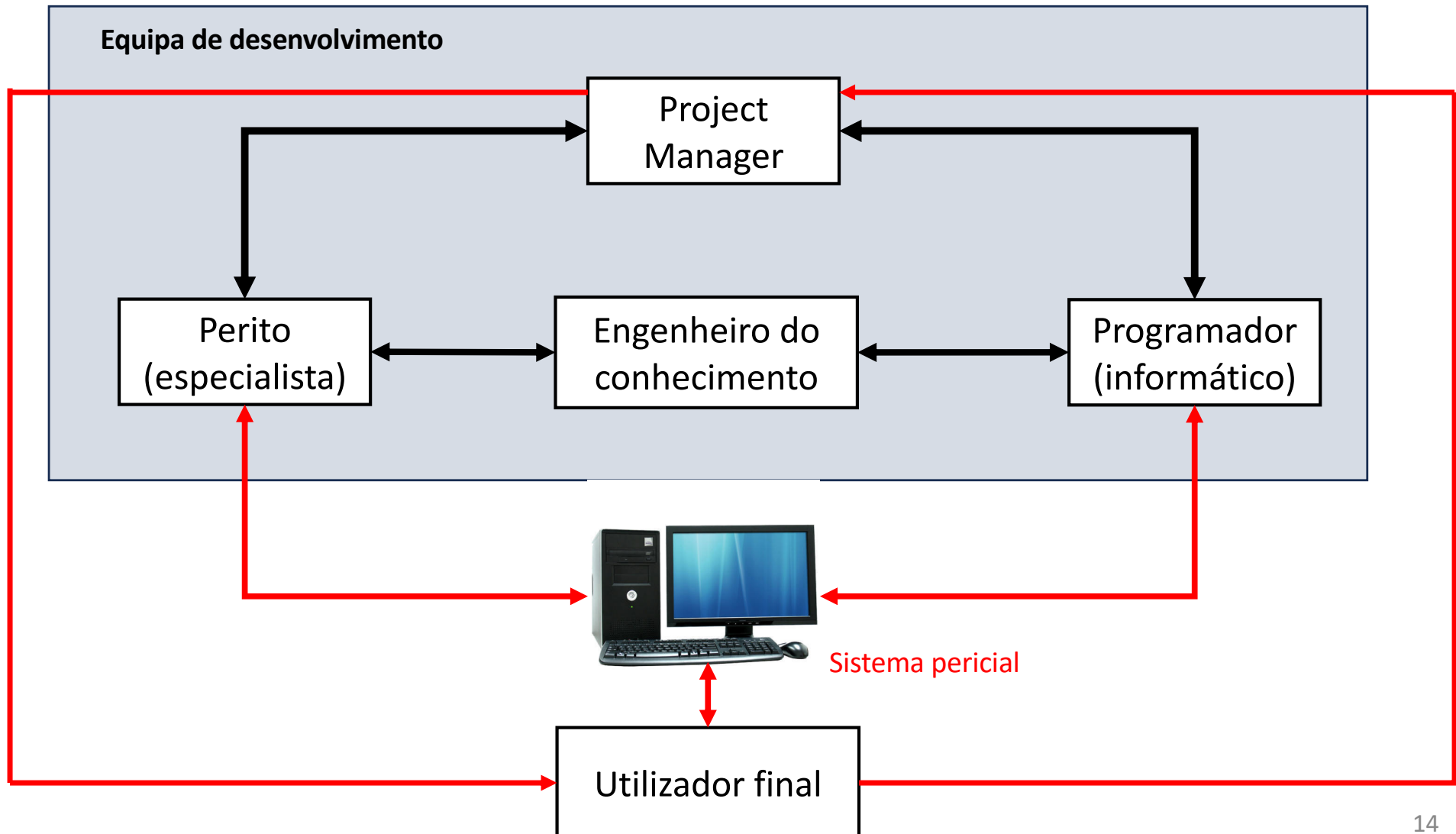
Sistemas Periciais – Aplicações reais

- **IBM**
Sistemas periciais para **diagnóstico e configuração** (ex.: apoio à decisão técnica, manutenção).
- **Siemens**
Diagnóstico de falhas em **sistemas industriais** e equipamentos médicos.
- **General Electric (GE)**
Sistemas periciais para **manutenção preditiva** em aviação e energia.
- **Oracle**
Motores de regras usados em **sistemas financeiros e de negócio**.
- **DXC Technology**
Sistemas periciais para **suporte à decisão** em seguros e saúde.

hoje muitos destes sistemas aparecem “disfarçados” como *rule engines* ou *decision systems*, mas a base é a mesma dos sistemas periciais clássicos.

Raciocínio Lógico

Sistemas Periciais – fluxograma



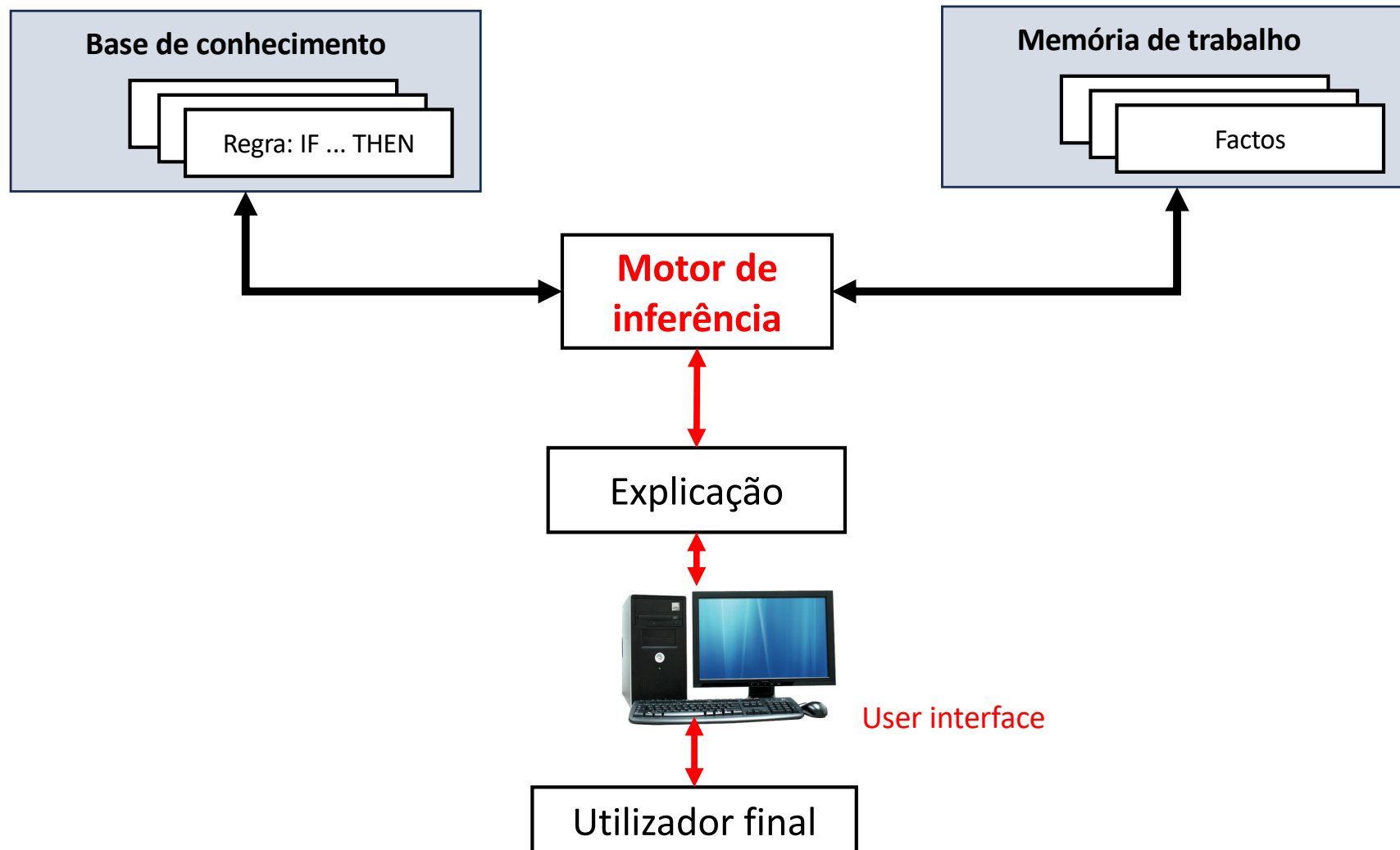
Raciocínio Lógico

Sistemas Periciais - componentes

- Base de conhecimento
 - Regras lógicas
 - Redes semânticas
 - Frames
 - Usada pelo motor de inferência
- Memória de trabalho
 - Factos conhecidos sobre o problema a resolver
 - Usada pelo motor de inferência
- Motor de inferência (raciocínio)
 - Infere as conclusões a partir dos factos (dados de entrada) e da base de conhecimento (regras)
 - Os factos são comparados com os antecedentes das regras e uma ou mais regras são ativadas, mostrando as ações dos respetivos consequentes.

Raciocínio Lógico

Sistemas Periciais



Raciocínio Lógico

Sistemas Periciais

- Motor de inferência
 - Realiza o raciocínio lógico
 - Permite que o sistema pericial resolva o problema
 - Faz a ligação das regras (base de conhecimento) aos factos (memória de trabalho)
 - Permite explicar e justificar como foi obtida a resposta:
 - Como é obtida a conclusão?
 - Porque razão é solicitada mais informação (novos factos) ao utilizador?

Raciocínio Lógico

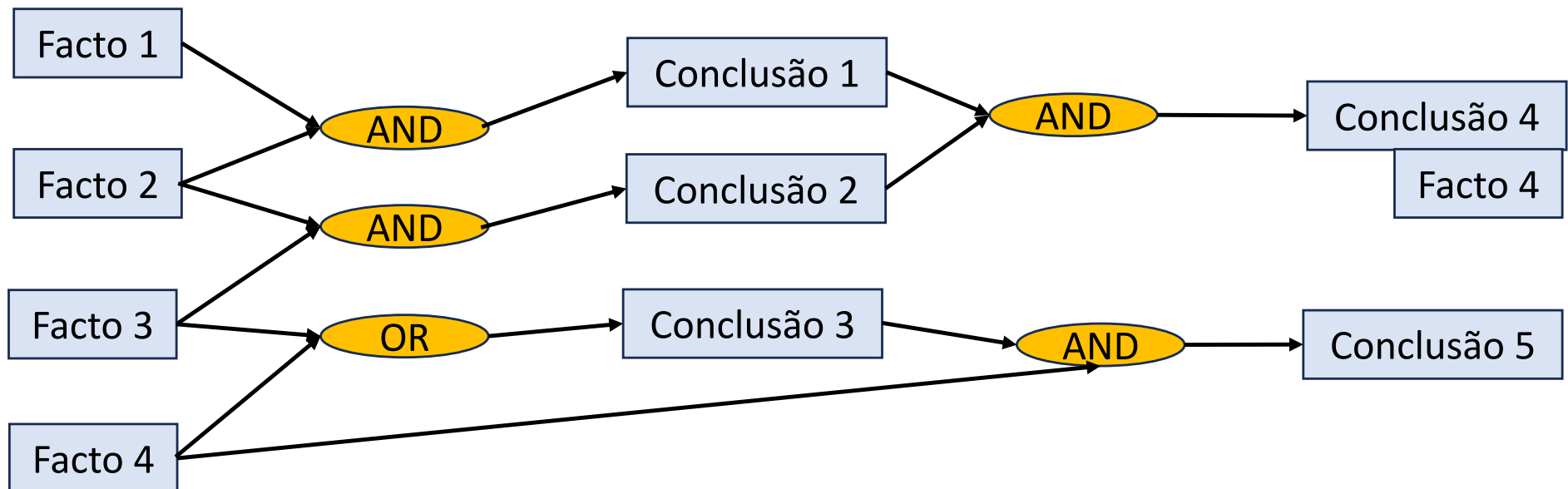
Sistemas Periciais

- Motor de inferência
 - Encadeamento para a frente (*forward chaining*)
 - Encadeamento para trás (*backward chaining*)
 - Raciocínio misto: usa ambos

Raciocínio Lógico

Sistemas Periciais

- Motor de inferência: *forward chaining*
 - Resolve o problema dos factos para inferir a conclusão
 - A conclusão pode passar a ser um novo facto da memória de trabalho, voltando a desencadear novas regras



Raciocínio Lógico

Sistemas Periciais - *forward chaining*



Patient has fever



Patient has body aches

Rules & Facts

- If a patient has a fever and body aches, they might have the flu.
- If a patient has a runny nose and cough, it could be a cold.
- If a patient has a fever, they might have an infection.

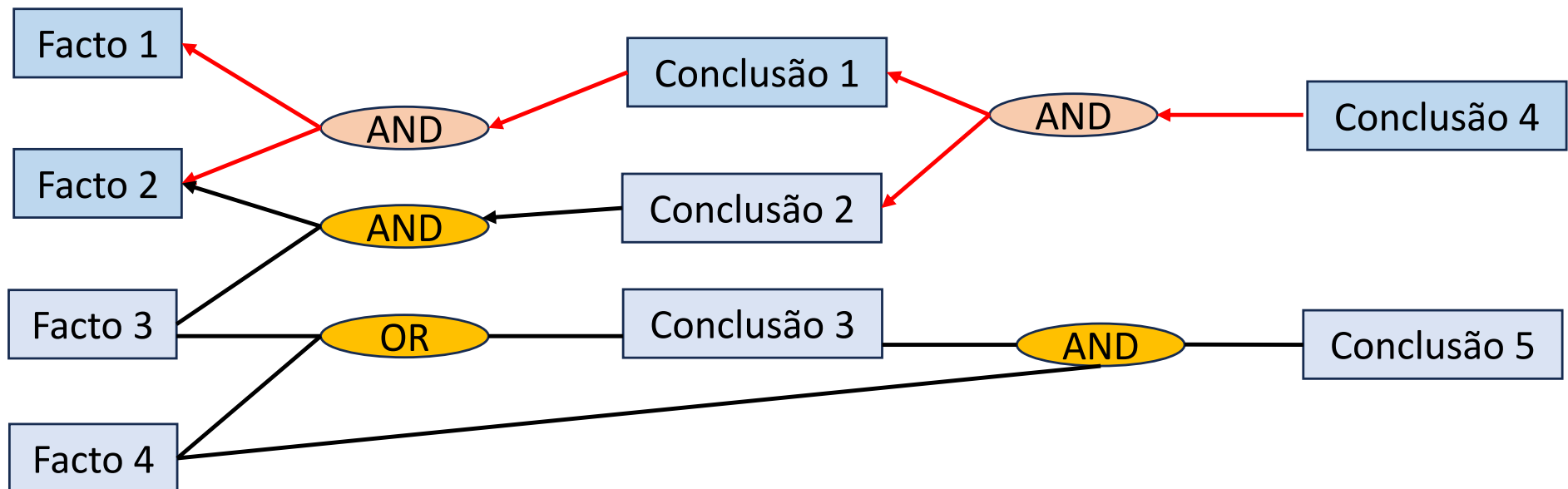
Forward Chaining Process

- Start with available facts: The patient has a fever and body aches.
- Rule 1: The patient has a fever and body aches, suggesting they might have the flu.
- This conclusion (might have the flu) becomes a new fact.

Raciocínio Lógico

Sistemas Periciais

- Motor de inferência: *backward chaining*
 - Resolve o problema a partir de uma conclusão conhecida
 - Procura na memória de trabalho os factos que expliquem a conclusão



Raciocínio Lógico

Sistemas Periciais - *backward chaining*



Diagnosis an Illness

Rules & Facts

- If a patient has a fever and body aches, they might have the flu.
- If a patient has a runny nose and cough, it could be a cold.
- If a patient has a fever, they might have an infection.
- If a patient has body aches, it might be due to physical exertion.

Backward Chaining Process

- Goal: The patient has the flu.
- Look at Rule 1: The patient has a fever and body aches.
- Look at Rule 3: The patient has a fever.
- Look at Rule 2: The patient has a runny nose and cough. This rule doesn't apply.
- Look at Rule 4: The patient has body aches.



If the patient has the Flu

Raciocínio Lógico

Sistemas Periciais

- Motor de inferência
 - Encadeamento para a frente ou direto (*forward chaining*)
 - Raciocínio guiado por factos
 - Analisam-se os factos existentes e executa-se apenas a regra mais ativa – adiciona mais um facto à memória de trabalho
 - Cada regra só pode ser executada uma vez
 - O ciclo termina quando mais nenhuma regra puder ser ativada.

Raciocínio Lógico

Sistemas Periciais

- Motor de inferência
 - Situações de conflito no *forward chaining* (*encadeamento para a frente*)
 - Ativar a regra com maior prioridade
 - Ativar a regra mais específica (a que têm mais informação)
 - Ativar a regra que tem associado os factos mais recentes da memória de trabalho

Raciocínio Lógico

Sistemas Periciais

- Motor de inferência
 - Encadeamento para trás ou inverso (*backward chaining*)
 - Raciocínio guiado por objetivos
 - Recebe um facto que deve ser demonstrado como verdadeiro e o motor de inferência procura evidências que provem a veracidade desse facto.
 - Procura as regras que têm esse facto como consequente e procura na memória de trabalho se há os antecedentes que provem a regra como verdadeira.
 - O ciclo termina quando não são encontradas mais regras onde exista o facto a provar como consequente.

Raciocínio Lógico

Sistemas Periciais

- Exemplo de *forward chaining*
- Regras:
 - R1: SE febre e dores e tosse ENTÃO gripe*
 - R2: SE febre e dores ENTÃO constipação*
 - R3: SE febre ENTÃO resfriado*
 - R4: Se gripe ENTÃO marcar consulta*

forward chaining

- Factos conhecidos F1 e F2 :
 - Ativam a regra R2 -> constipação*
- Factos conhecidos :
 - Ativam a regra R3 -> resfriado*

- Factos iniciais:

F1: febre

F2: dores

F3: tosse

- Novos factos:

F4: constipação

F5: resfriado

Raciocínio Lógico

Sistemas Periciais

- Exemplo de *backward chaining*

- Regras:

R1: SE febre e dores e tosse ENTÃO gripe

R2: SE febre e dores ENTÃO constipação

R3: SE febre ENTÃO resfriado

- Diagnóstico: O paciente tem gripe:

Procurar na memória de trabalho se há factos que coloquem a regra R1 verdadeira para Maria

- *F3 não existe, não ativa a regra R1*

- Diagnóstico: paciente tem constipação:

Procurar na memória de trabalho se há factos que coloquem a regra R2 verdadeira

- *F1 e F2 ativam a regra R2*

- Factos iniciais:

F1: febre

F2: dores

Raciocínio Lógico

Sistemas Periciais

- Exemplo de *forward chaining*

- Regras:

R1: SE febre e dores e tosse ENTÃO gripe

R2: SE febre e dores ENTÃO constipação

R3: SE febre ENTÃO resfriado

R4: Se gripe ENTÃO marcar consulta

forward chaining

- Factos conhecidos F1 e F2 para João:

Ativam a regra R2 -> João tem constipação

- Factos conhecidos F3 para Maria:

Ativam a regra R3 -> Maria tem resfriado

- Factos conhecidos F4 para Diana:

Ativam a regra R3 -> Diana tem resfriado

- Factos conhecidos F4, F5, F6 para Ana:

*Ativam a regra R1 -> Ana tem **gripe***

- Factos iniciais:

F1: João tem febre

F2: João tem dores

F3: Maria tem febre

F4: Diana tem febre

F5: Ana tem febre

F6: Ana tem dores

F7: Ana tem tosse

- R1 gera um novo facto para Ana

*F8: Ana tem **gripe***

- F8 ativa a regra R4

R4: Marcar consulta para Ana

Raciocínio Lógico

Sistemas Periciais

- Exemplo de *backward chaining*

- Regras:

R1: SE febre e dores e tosse ENTÃO gripe

R2: SE febre e dores ENTÃO constipação

R3: SE febre ENTÃO resfriado

- Diagnóstico: A Ana tem gripe:

Procurar na memória de trabalho se há factos que coloquem a regra R1 verdadeira para Ana

- F4, F5 e F6 ativam a regra R1

- Diagnóstico: O João tem constipação:

Procurar na memória de trabalho se há factos que coloquem a regra R2 verdadeira para João

- F1 e F2 ativam a regra R2

- Diagnóstico: O Maria tem constipação:

Procurar na memória de trabalho se há factos que coloquem a regra R2 verdadeira para Maria

- F3 não ativa a regra R2

- Factos:

F1: João tem febre

F2: João tem dores

F3: Maria tem febre

F4: Ana tem febre

F5: Ana tem dores

F6: Ana tem tosse

Raciocínio Lógico

Sistemas Periciais – exercício 1

Considere a base de conhecimento seguinte:

- **Factos**

- F1: temperatura_alta
- F2: ventilador_desligado

- **Regras**

- R1: SE temperatura_alta E ventilador_desligado ENTÃO risco_sobreaquecimento
- R2: SE risco_sobreaquecimento ENTÃO emitir_alarme

PERGUNTAS

- a. Indique todos os novos factos inferidos pelo sistema usando forward chaining.
- b. Mostre a sequência de aplicação das regras.
- c. Qual é o estado final da base de factos?

Raciocínio Lógico

Sistemas Periciais – exercício 1

Resolução:

- **Factos iniciais**
 - temperatura_alta
 - ventilador_desligado
- **Inferência**
 - R1 é activada → risco_sobreaquecimento
 - Insere novo facto -> risco_sobreaquecimento. Logo:
 - R2 é activada → emitir_alarme
- **Factos finais**
 - temperatura_alta
 - ventilador_desligado
 - risco_sobreaquecimento
 - emitir_alarme

Raciocínio Lógico

Sistemas Periciais – exercício 2

Considere a base de conhecimento seguinte:

- **Factos**

- F1: temperatura_alta
- F2: ventilador_desligado

- **Regras**

- R1: SE temperatura_alta E ventilador_desligado ENTÃO risco_sobreaquecimento
- R2: SE risco_sobreaquecimento ENTÃO emitir_alarme

PERGUNTAS

- Considere o seguinte objetivo G: **emitir_alarme?**
 - a. Explique passo a passo como o sistema prova o objetivo usando backward chaining.
 - b. Que factos precisam de ser verificados?
 - c. O objetivo é satisfeito? Justifique.

Raciocínio Lógico

Sistemas Periciais – exercício 2

Resolução:

- **Factos iniciais**

- temperatura_alta
- ventilador_desligado

- **G: emitir_alarme?**

- **Raciocínio**

- Para provar emitir_alarme, aplica-se R2
→ precisa de risco_sobreaquecimento
- Para provar risco_sobreaquecimento, aplica-se R1
→ precisa de temperatura_alta E ventilador_desligado
- Ambos são verdadeiros

✓ **Objetivo satisfeito**

Raciocínio Lógico

Sistemas Periciais – exercício 3

Considere a base de conhecimento seguinte:

- **Factos**

- F1: febre
- F2: dor_garganta
- F3: ganglios_inchados

- **Regras**

- R1: SE febre E dor_garganta ENTÃO infeção
- R2: SE infeção E ganglios_inchados ENTÃO suspeita_amigdalite
- R3: SE suspeita_amigdalite ENTÃO recomendar_consulta

- **PERGUNTAS**

- a. Liste os factos inferidos por forward chaining.
- b. Indique que regras são ativadas e porquê.
- c. Qual a conclusão final do sistema?

Raciocínio Lógico

Sistemas Periciais – exercício 3

Resolução:

- **Factos iniciais**

- febre
- dor_garganta
- ganglios_inchados

- **Inferência**

- Com **febre** R1 é activada → infeção

Inserir novo facto -> infeção. Logo:

- R2 é ativada → suspeita_amigdalite

Inserir novo facto -> suspeita_amigdalite. Logo:

- R3 é ativada → recomendar_consulta

- **Conclusão final**

- recomendar_consulta

Raciocínio Lógico

Sistemas Periciais – exercício 4

Considere a base de conhecimento seguinte:

- **Factos**

- F1: rendimento_estavel = verdadeiro

- F2: historico_credito_positivo = verdadeiro

- F3: idade_maior_21 = verdadeiro

- **Regras**

- R1: SE rendimento_estavel E historico_credito_positivo ENTÃO cliente_baixo_risco

- R2: SE cliente_baixo_risco E idade_maior_21 ENTÃO credito_aprovado

PERGUNTAS

- Considere o seguinte objetivo G: credito_aprovado?
 - a. Descreva o raciocínio do sistema usando backward chaining.
 - b. Que sub-objetivos são gerados?
 - c. O crédito é aprovado? Explique.

Raciocínio Lógico

Sistemas Periciais – exercício 4

Resolução:

- **Factos iniciais**

F1: rendimento_estavel = verdadeiro

F2: historico_credito_positivo = verdadeiro

F3: idade_maior_21 = verdadeiro

- G: credito_aprovado

- **Raciocínio**

- Para provar **credito_aprovado**, aplica-se R2

R2 → precisa de **cliente_baixo_risco** E **idade_maior_21**
idade_maior_21 é verdadeiro

- Para **cliente_baixo_risco**, aplica-se R1

→ rendimento_estavel e historico_credito_positivo

- Ambos verdadeiros

✓ **Crédito aprovado**

Raciocínio Lógico

MYCIN – fatores de certeza nas regras

- O **MYCIN** foi concebido como um **sistema *goal-driven***:
 - Parte de um **objetivo** (ex. determinar a causa)
 - Procura regras que possam concluir esse objetivo
 - Gera perguntas ao utilizador apenas quando necessário
- Usa backward chaining (encadeamento para trás)
- Usa **Fatores de certeza (CF)** em cada regra, para lidar com a incerteza em alguns domínios de conhecimento

Raciocínio Lógico

MYCIN – fatores de certeza nas regras

- **Tarefas:**

- Identificar o **objetivo** (problema?)
- Aplicar **encadeamento para trás**
- Indicar:
 - que regras foram ativadas
 - que perguntas foram feitas ao utilizador
 - conclusão + fator de certeza

Raciocínio Lógico

MYCIN – fatores de certeza nas regras

- No **MYCIN**, o **fator de certeza (CF)** não é probabilístico. É uma heurística definida pelos peritos.

SE condições ENTÃO hipótese [CF_regra]

Exemplo Se **A** e **B**, então **H** [CF = 0.7]

- **CF das condições**

- Se uma condição é **facto observado** o fator de certeza é 1 → ex. CF(A) = 1
- Senão é fornecido o CF de uma determinada condição → ex. CF(B) = 0.5
- Se é inferida por outra regra → usa o CF dessa inferência

Raciocínio Lógico

MYCIN – fatores de certeza nas regras

- Calcular fatores de certeza das condições das regras

Operador	CF
AND	$\min(CF_1, CF_2, \dots)$
OR	$\max(CF_1, CF_2, \dots)$
NOT	$-CF$

- Calcular fatores de certeza da conclusão de uma regra
 - $CF_{\text{conclusão}} = CF_{\text{condições}} \times CF_{\text{regra}}$

Raciocínio Lógico

MYCIN – fatores de certeza nas regras

- Se várias regras concluírem a mesma hipótese?
- Quando mais de duas regras suportam a mesma hipótese, o MYCIN combina os fatores de certeza de forma incremental, aplicando repetidamente as fórmulas binárias de combinação.
- Para 2 regras
 - Se ambos CF são **positivos** (reforço):
 - $CF_{\text{combinado}} = CF_1 + CF_2 \times (1 - CF_1)$
 - Se ambos CF são **negativos** (reforço contra)
 - $CF_{\text{combinado}} = CF_1 + CF_2 \times (1 + CF_1)$
 - **Sinais opostos** (evidência a favor e contra)
 - $CF = (CF_1 + CF_2) / (1 - \min(|CF_1|, |CF_2|))$

Raciocínio Lógico

MYCIN – fatores de certeza nas regras

- No **MYCIN**, quando mais de 2 regras concluem a mesma hipótese:
 - Cada regra produz um CF parcial
 - Os CFs são combinados **dois a dois**
 - O resultado intermédio é combinado com o seguinte
 - Repete até não haver mais evidência
- Formalmente:

$CF_final = combine(combine(CF1, CF2), CF3), \dots$

Raciocínio Lógico

MYCIN – fatores de certeza nas regras

- Resumindo

Situação	Fórmula
$CF_1 \geq 0$ e $CF_2 \geq 0$	$CF_1 + CF_2(1 - CF_1)$
$CF_1 \leq 0$ e $CF_2 \leq 0$	$CF_1 + CF_2(1 + CF_1)$
Sinais opostos	$(CF_1 + CF_2)/(1 - \min(CF_1 , CF_2))$

- Mesmo sinal → **reforço**
- Sinais opostos → **compensação**
- Nunca ultrapassa ± 1
- Não são probabilidades, é **conhecimento heurístico**

Raciocínio Lógico

MYCIN – fatores de certeza nas regras

- Exemplo 1 (**ambos positivos**)

$$CF_1 = 0.6, CF_2 = 0.5$$

$$CF_{\text{final}} = 0.6 + 0.5 \times 0.4 = \mathbf{0.8}$$

- Exemplo 2 (**ambos negativos**)

$$CF_1 = -0.6, CF_2 = -0.5$$

$$CF_{\text{final}} = -0.6 + (-0.5) \times (1 - 0.6) = -0.6 - 0.2 = \mathbf{-0.8}$$

- Exemplo 3 (**sinais opostos**)

$$CF_1 = 0.7, CF_2 = -0.4$$

$$CF = (0.7 - 0.4) / (1 - 0.4) = 0.3 / 0.6 = \mathbf{0.5}$$

- Exemplo 4 (**combinar mais de 2 regras**)

$$CF_1 = 0.6, CF_2 = 0.5, CF_3 = -0.4$$

Passo 1 — combinar R1 e R2 (ambos positivos)

$$CF_{12} = 0.6 + 0.5 \times (1 - 0.6) = 0.6 + 0.2 = \mathbf{0.8}$$

Passo 2 — combinar com R3 (sinais opostos)

$$CF_{\text{final}} = (0.8 - 0.4) / (1 - 0.4) = 0.4 / 0.6 \approx \mathbf{0.67}$$

Raciocínio Lógico

MYCIN – fatores de certeza nas regras

SE a estirpe é gram-positiva E a morfologia é coccus E a multiplicação é em cadeias
ENTÃO streptococcus [CF=0.7]

$CF_{\text{estirpe}}=0.5$ (estirpe é conhecida com certeza 0.5)
 $CF_{\text{morfologia}}=0.6$ (morfologia é conhecida com certeza 0.6)
 $CF_{\text{cadeias}}=0.3$ (multiplicação em cadeias é conhecida com certeza 0.3)

Passo 1 – CF das condições

$$CF_{\text{condições}} = \min(CF_{\text{estirpe}}, CF_{\text{morfologia}}, CF_{\text{cadeias}}) = \min(0.5, 0.6, 0.3) = 0.3$$

Passo 2 - CF_conclusão = CFcondições × CFregra

$$CF_{\text{final}} = 0.3 \times 0.7 = 0.21$$

Suponhamos agora que uma regra R2 também conclui com streptococcus e $CF_2=0.5$.

Como ambos os CF são positivos, o CF da conclusão final acerca de se tratar de streptococcus é

$$CF_{\text{final}} = CF_1 + CF_2(1 - CF_1) = 0.21 + (0.5 \times (1 - 0.21)) = 0.605$$

Raciocínio Lógico

MYCIN – Exercício 1

R1: SE luz_power = desligada ENTÃO problema = "sem alimentação" [CF = 0.9]

R2: SE luz_power = ligada E luz_internet = desligada ENTÃO problema = "falha ISP" [CF = 0.8]

Observações (evidência):

- luz_power = ligada $\rightarrow$ CF = 1
- luz_internet = desligada $\rightarrow$ CF = 0.7

Qual a falha detetada?

Raciocínio Lógico

MYCIN – Exercício 1

Resolução

R1: luz_power ligada $\rightarrow$ falha (condição falsa) $\rightarrow$ ignora R1

R2: AND $\rightarrow$ CF_condições = $\min(1, 0.7) = 0.7$

CF_conclusão = CF_condições $\times$ CF_regra = $0.7 \times 0.8 = 0.56$

Resultado final:

problema = "falha ISP" [CF = 0.56]

Raciocínio Lógico

MYCIN – Exercício 2

R1: SE luz_wifi = desligada ENTÃO problema = wifi desligado [CF = 0.9]

R2: SE luz_internet = desligada ENTÃO problema = internet desligada [CF = 0.8]

Observações (evidências):

- luz_wifi = ligada $\rightarrow$ CF = 1
- luz_internet = desligada $\rightarrow$ CF = 0.6

Hipótese: internet indisponível = R1 OR R2

Qual o resultado final para esta hipótese?

Raciocínio Lógico

MYCIN – Exercício 2

Resolução

- **Passo 1 — Calcular CF Regra 1 e Regra 2**
 - **R1:** condição não satisfeita $\rightarrow CF = 0$
 - **R2:** condição satisfeita $\rightarrow CF = 0.6 \times 0.8 = 0.48$
- **Passo 2 — Combinar regras R1 e R2 usando OR**
 - $R1 \rightarrow CF = 0$
 - $R2 \rightarrow CF = 0.48$
 - $OR \rightarrow \text{máximo: } CF_{\text{final}} = \max(0, 0.48) = 0.48$

Resultado final: internet indisponível [CF = 0.48]

Raciocínio Lógico

MYCIN – Exercício 3

R1: SE sensor_fogo = ligado ENTÃO alerta = "fogo detetado" [CF = 0.9]

R2: SE sensor_fumo = ligado OU sensor_calor = ligado ENTÃO alerta = "fogo detetado" [CF = 0.8]

- **Observações:**

- sensor_fogo = desligado $\rightarrow$ CF = 0
- sensor_fumo = ligado $\rightarrow$ CF = 0.7
- sensor_calor = desligado $\rightarrow$ CF = 1

Qual o fator de certeza para a Hipótese: alerta = "fogo detetado"

Raciocínio Lógico

MYCIN – Exercício 3

Resolução

R1: condição não satisfeita $\rightarrow CF = 0$

R2: OR $\rightarrow CF_{condições} = \max(0.7, 0) = 0.7$

$CF_{final} = 0.7 \times 0.8 = 0.56$

Resultado final:

alerta = "fogo detectado" [CF = 0.56]

Raciocínio Lógico

MYCIN – Exercício 4

R1: SE luz_power = desligada ENTÃO problema = "sem alimentação" [CF = 0.9]

R2: SE luz_internet = desligada ENTÃO problema = "falha ISP" [CF = 0.8]

R3: SE NOT luz_wifi = ligada ENTÃO problema = "wifi desactivado" [CF = 0.7]

- Observações:
- luz_power = ligada $\rightarrow$ CF = 1
- luz_internet = desligada $\rightarrow$ CF = 0.6
- luz_wifi = ligada $\rightarrow$ CF(NOT luz_wifi) = -1

Hipótese: problema de conectividade = R1 OR R2 OR R3

Qual o resultado final para esta hipótese?

Raciocínio Lógico

MYCIN – Exercício 4

Resolução

- $R1 \rightarrow$ condição falsa $\rightarrow$ ignora
- $R2 \rightarrow CF = 0.6 \times 0.8 = 0.48$
- $R3 \rightarrow \rightarrow$ condição “negada” é falsa $\rightarrow$ ignora
- Combinação OR $\rightarrow$ máximo = **0.48**

Resultado final:

problema de conectividade = 0.48

Raciocínio Lógico

Sistemas Periciais

- Vantagens dos sistemas periciais
 - Capacidade de armazenamento permanente do conhecimento (não há esquecimento)
 - Respostas consistentes ao mesmo problema
 - No mesmo sistema é possível combinar conhecimento de mais do que um especialista
 - Economicamente mais vantajosos
 - Utilização fácil e intuitiva para o utilizador final porque imitam o raciocínio humano

Raciocínio Lógico

Sistemas Periciais

- Desvantagens dos sistemas periciais
 - Aquisição do conhecimento do especialista:
 - tarefa morosa, subjetiva, e muitas vezes pouco eficiente
 - Erros no sistema de regras podem ter resultados catastróficos
 - Um erro num diagnóstico médico por exemplo
 - A fase de treino e teste até à sua utilização é muito demorada
 - Sistemas pouco criativos, sem capacidade de aprendizagem
 - Novo conhecimento tem de ser introduzido
 - Em muitos domínios, o sistema precisa de milhares de regras:
 - Pouca versatilidade na correção de erros e atualização destes sistemas